

Distributed System Assignment 1
Hsiao, Po-Hung
1719722

1. System Overview

/src/Server

DictionaryServerGUI.java:

The entry point for the server application. It contains the main method and parses command-line arguments. It provides a graphical interface to start or stop the server and monitor the server's connection status, active connections, and log records.

DictionaryServer.java:

This class initialised a serverSocket to listen to a specific port for incoming connection requests. Using the thread-per-connection model, it creates a new thread for every successful connection, allowing the main thread to immediately return to listening for other clients.

DictionaryManager.java:

This class uses a ConcurrentHashMap to maintain word definitions in memory and manages the loading and saving of dictionary data to a persistent TXT file. It is also responsible for all operation logic with the dictionary data. Before every operation, it uses the ReadWriteLock to ensure the data concurrency.

ServerHandler.java:

This class implements the Runnable interface. Each instance runs in a dedicated thread to manage a specific client's socket I/O. It parses incoming JSON messages, invokes the appropriate operation logic in the manager, and sends the result back to the client, ensuring that multiple clients are processed concurrently.

src/Client

DictionaryClientGUI.java:

This class is the entry for the client application, containing the main method which reads command-line arguments. It provides a graphical user interface, capturing the user action and passing it to the DictionaryClient. Then, it waits for the DictionaryClient to send back server responses and updates the result in the user interface.

DictionaryClient.java:

This class is responsible for the communication between client and server. It establishes the TCP socket to connect with the server and manages the input/output streams. It parses the DictionaryMessage object to JSON and sends it to the server, then receives the response and parses it back into the object.

/src/Common

DictionaryMessage.java

A standardized data transfer object shared between the client and server. It encapsulates the request details into a structured format, which is then transformed into JSON strings using the GSON library for network transmission.

/lib/gson-2.11.0.jar

2. Class Design And Interactive Diagram

2.1 UML Diagram

UML Link:

<https://drive.google.com/file/d/17uninTaSr9aOnM80fHb734fx8cC4pcST/view?usp=sharing>

The UML diagram illustrates an architecture organised into three logical packages: Common, Server, and Client.

In the client package, there is a composition relationship between DictionaryClientGUI and DictionaryClient. The GUI class manages the DictionaryClient instance. This design separates the UI from the network logic. It also ensures the connection closes safely when the user exits.

In the server package, there is also a composition relationship between DictionaryServerGUI and DictionaryServer. And there is an aggregation relationship between DictionaryServer and ServerHandler. The DictionaryServer is a listener that can generate several new ServerHandler instances for every successful connection. Furthermore, there is an association between ServerHandler and DictionaryManager. Each thread holds a reference to the manager to invoke operation logic.

In the common package, DictionaryMessage, as a data transfer object, has a dependency relationship with DictionaryClient and ServerHandler.

2.2 Interactive Diagram

Interactive Diagram Link:

<https://drive.google.com/file/d/1DagmACCVheJrqVKbft4TGjdA6P0kA0Ww/view?usp=sharing>

The interaction diagram illustrates the workflow of adding a word in the dictionary, focusing on socket communication and thread management.

First, the user inputs arguments to start the application. DictionaryClient establishes a TCP connection to the server using connect(), which creates a socket bound to the server's address and port. This connection remains open throughout the session, enabling continuous request-response communication.

On the server side, the DictionaryServer is initialized by the DictionaryServerGUI through startServer(). Once started, the server enters a continuous blocking loop, repeatedly invoking serverSocket.accept() to listen for incoming client connections. When a client connects, a clientSocket is created, and the server immediately creates a new thread to handle that specific client (clientThread.start()). This design follows a thread-per-connection model, allowing multiple clients to interact with the server concurrently.

When the user clicks the “Add” button, the GUI invokes `addWord()` and then executes `executeRequire()` to pass the `DictionaryMessage`. Each client thread is managed by a `ServerHandler`. `ServerHandler` continuously listens until receiving a request, such as adding a word. The handler forwards the operation to the `DictionaryManager`, which encapsulates the core operation logic and shared dictionary data.

To ensure concurrent access, a write lock is applied before modifying the dictionary. This guarantees that only one thread can modify data at a time. The `sleep()` simulates processing time and helps test the concurrency mechanism under contention.

After the operation, the result is encapsulated into a response message and sent back to the client through the socket. The `DictionaryClient` receives this response and passes it to the `DictionaryClientGUI`. Finally, the result is displayed on the user’s interface.

3.1 Why is TCP over UDP chosen or vice versa in your design? Explain

Reliability is a critical factor of the dictionary. TCP provides mechanisms for error detection and packet loss retransmission to ensure the user can get the complete answer. In addition, TCP also ensures the ordering of data, which is important for the correction of the deletion function and update function. Finally, TCP is connection-oriented, which can easily detect the failure connection of the client by “`reader.readLine()`”. While UDP offers low latency, building a reliable mechanism by UDP would take much time and increase the complexity of development.

3.2 Why is a specific messaging exchange protocol, e.g., XML, used over others?

Explain

I use JSON as my message exchange protocol. The main reason is JSON’s structure is highly aligned with Java. JSON represents data as key-value pairs, which matches Java’s `map` interface and the field names / values of Java objects. In addition, JSON allows the nested structure, so the value can store the data type as `List<String>` when using the add function. Finally, JSON has an easier parsing tool like `Gson`, while XML needs a more complex parsing method. Compared to custom text protocols, structural protocols are more stable; for example, JSON has built-in escaping mechanisms to ensure data integrity.

3.3 How did you ensure or implement the “concurrent READS and restrict concurrent WRITES?” Explain

I choose `reentrantReadWriteLock` to implement the concurrent requirement. Rather than `Synchronized` method, `reentrantReadWriteLock` has read locks and write locks, which is better for the product as a dictionary, which is queried often.

Before querying the data, lock a read lock to block the thread that wants to modify the data; just allow the thread which queries the data. Before modifying the data, lock the write lock to make every thread wait until the lock unlocks. Furthermore, I use the `try...finally` block to avoid the deadlock; no matter if the errors or exceptions occur, the lock will be unlocked eventually.

3.4 What are the pros/cons of the chosen architecture (client-server) and thread model (thread-per-object or thread-per-connection or thread-pool) or thread-safety mechanism (allow concurrent READS and disable concurrent WRITES)?

Centralizing management of data in the server is one of the pros of client-server architecture. It makes the data retrieval consistent from every client. Also, this architecture has great security because the client can only modify data by sending the request to the server. However, a single point of failure is the main con of this architecture. The system highly relies on the server; if the server crashes or disconnects, the whole system will break down.

I use thread-per-connection as my thread model. It has clear and easy connection logic: one client socket matches one thread. For the above reason, this connection has low latency, as every thread has their own socket. However, if there are a thousand client connections, it means there are thousands of threads, making high resource overhead. So thread-per-connection is suitable for the small project, which can balance the latency and resource consumption.

I choose to implement `reentrantReadWriteLock` on concurrent requirements. This mechanism makes many threads query at the same time to enhance read throughput. It also ensures the data integrity by using an exclusive lock. But there is a notable con: writer starvation, which means if there are many queries, the server will be locked by a read lock, and then the write lock will have no time to execute its work.

3.5 What is the purpose of introducing an intended delay or sleep argument in the server operations? Does this affect the design choices made? Explain how?

The main purpose of introducing a delay is to validate the lock implementation. Because concurrent operations on a local machine would complete too rapidly to observe the locking behaviour. According to the delay time, we can adjust the lock granularity if the server operates too slowly. Furthermore, an intended delay highlights the pro of `reentrantReadWriteLock` rather than `Synchronised` method. Using `reentrantReadWriteLock` can make many threads query the dictionary without much delay.